

Web-Based Automation of Student Identity Card Photo Processing Using AI Background Removal

First Author¹, Second Author²

^{1,2} Department of Informatics, Faculty of Engineering, Universitas [Institution Name], Jakarta, Indonesia
e-mail: author1@email.ac.id, author2@email.ac.id

Accepted on mmmmm dd, yyyy

Approved on mmmmm dd, yyyy

Abstract—The production of student identity cards (Kartu Tanda Mahasiswa, KTM) in many higher-education institutions is still handled manually: photo backgrounds are removed one by one with image-editing software, file naming is inconsistent, and files are archived without a clear structure. This paper presents the design and implementation of a web-based system that automates the KTM photo-processing workflow end to end. The system integrates AI-based background removal built on the Rembg library and the U2-Net salient object detection architecture, real-time photo capture through the browser, automatic retrieval of student data from the campus academic information system, dynamic composition of the photo onto undergraduate or graduate card templates, and structured archiving in a year/month/study-program directory hierarchy with student-number-based file naming. A gallery module provides listing, filtering, deletion, and bulk ZIP export of archived files for card printing. The system is implemented as a hybrid Next.js and Express.js application that invokes the U2-Net model through ONNX Runtime on commodity CPU hardware. Testing on 100 sample photos shows that the system reduces per-student processing time from roughly 3-5 minutes (manual) to under 6 seconds (automatic), with 86% of samples segmented cleanly without any manual correction. The proposed system improves the efficiency, consistency, and traceability of KTM photo management.

Index Terms—background removal; image segmentation; student identity card; U2-Net; web-based system; workflow automation

I. INTRODUCTION

A student identity card (Kartu Tanda Mahasiswa, KTM) is the official identity document issued by a higher-education institution to every registered student. It serves as an administrative prerequisite for a wide range of academic services, such as library borrowing, examinations, laboratory access, and the use of campus facilities. One essential component of the card is the student photograph, which must appear neat, uniform in size and position, and placed on a clean, standardized background. In practice, however, the production and management of KTM in many institutions is still performed manually, in which student data and photographs are processed one by one; such workflows are error-prone and time-consuming [1].

The problems of the manual workflow can be broken down into three parts. First, background removal is carried out individually using general-purpose image editors such as Adobe Photoshop or GIMP, which demands operator skill and considerable time, especially when the number of new students reaches the thousands in every admission cycle. Second, file naming is frequently inconsistent from one operator to another, which makes it difficult to retrieve a specific photograph afterwards. Third, the resulting files are not organized in a predictable storage structure, so the traceability of the photo archive is low. An additional, often overlooked source of error is the manual (re)typing of student identity data—student number, name, and study program—during photo sessions, which can produce mismatches between the printed card and the academic database.

Advances in computer vision have made fully automatic background removal practical. Semantic segmentation with fully convolutional networks [2] and the encoder-decoder U-Net architecture, originally proposed for biomedical image segmentation [3], laid the foundation for dense pixel-level prediction. A closely related task, salient object detection (SOD), aims to segment the object that most attracts human visual attention from its background and has been studied extensively [4], [5], and deep architectures now dominate the field of image segmentation in general [6]. Qin et al. [7] proposed U2-Net, a two-level nested U-structure that captures both fine local detail and global context without relying on a classification backbone, achieving strong SOD performance at modest computational cost. U2-Net is the default engine of the open-source Rembg library [8], which exposes automatic background removal through a simple command-line and Python interface and executes the network through the ONNX Runtime inference engine [9], entirely on local hardware without any cloud dependency.

In the Indonesian academic context, digital image processing has been applied to student-related services in several ways, for example face recognition of students using PCA and Viola-Jones as part of biometric databases [10] and optical character recognition for extracting information from KTM images [1]. These studies concentrate on the recognition side, i.e., extracting information from images that already exist. Research that specifically

addresses the production side—integrating AI-based background removal with automated naming, template composition, and structured archival of KTM photographs in a single ready-to-use workflow—remains limited. Furthermore, most available background-removal tools are either desktop utilities operated per file or paid cloud services that raise privacy concerns when applied to student personal data.

Based on this background, this research designs and builds a web-based system with the following objectives: (1) to remove the background of newly captured student photographs automatically using AI background removal; (2) to capture photographs directly in the browser with a live camera preview, or alternatively accept uploaded image files; (3) to retrieve student identity data (student number, name, and study program) automatically from the campus academic information system so that no manual typing is required; (4) to compose the processed photograph onto the appropriate card template (undergraduate or graduate) together with the student identity text; (5) to archive the processed photograph with a consistent naming convention inside a hierarchical year/month/study-program directory structure; and (6) to provide a gallery module for browsing, filtering, deleting, and bulk-exporting archived files as a ZIP package for mass card printing. The main contribution of this work is the end-to-end integration of AI-based image processing and structured archive management in a single web workflow that runs fully on-premises, which distinguishes it from per-file desktop tools and from recognition-oriented prior work.

The remainder of this paper is organized as follows. Section II describes the development method, system architecture, processing pipeline, and testing scenario. Section III presents the implementation results, quantitative and functional test results, and a discussion of strengths and limitations. Section IV concludes the paper and outlines future work.

II. METHOD

This research follows a software prototyping approach consisting of requirement analysis, system design, implementation, and testing. Requirements were gathered from the academic services workflow for new-student onboarding, in which hundreds to thousands of students must be photographed, processed, and issued cards within a short period. The prototype was refined iteratively based on trial runs of the photo-capture workflow.

A. System Architecture

The system is implemented as a hybrid web application composed of two cooperating server processes. The first is a Next.js 15 application (App Router, React 19) that serves the user interface and a set of internal API routes for orchestration. The second is an Express.js worker service that handles heavy processing and static file serving in production deployments. Both processes are managed by the PM2

process manager and communicate over HTTP following REST architectural principles [11]. The AI engine itself is the Rembg command-line tool [8], which loads the U2-Net model [7] through ONNX Runtime [9] and is invoked by the server as an external process. Raster operations before and after inference—resizing, blurring, and median filtering—are performed with the sharp image-processing library [12]. Access to the application is restricted by a token-based login middleware, so only authorized operators can capture, archive, or delete photographs.

Two design decisions in the architecture deserve emphasis. First, all image processing runs locally on the institution's own server; no student photograph ever leaves the campus infrastructure, which is important because student photographs and identity data are personal data. Second, archived images are never served directly from the file system port; instead, an internal image-proxy API route streams files through the web framework [13]. This makes the gallery robust against reverse-proxy and port-forwarding configurations commonly found in campus server environments and avoids cross-origin issues between the two server processes.

TABLE I. SOFTWARE COMPONENTS AND THEIR ROLES IN THE SYSTEM

Component	Role in the System	Base
Next.js 15	User interface, API orchestration, image proxy, gallery	React 19
Express.js	Worker service for processing and static file serving	Node.js
Rembg	Automatic background removal (CLI)	U2-Net
ONNX Runtime	Inference engine executing the U2-Net model	ONNX
sharp	Resizing, blur, and median filtering	libvips
Multer	Multipart upload handling	Node.js
SIS REST API	Source of student identity data	Campus SIS
PM2	Process management and production deployment	Node.js

B. Photo-Processing Pipeline

The system processes every photograph through six consecutive stages, and the core server-side logic is summarized in Fig. 1.

1) Image acquisition and data binding. The operator opens the capture page, which streams the device camera at a 3:4 aspect ratio using the browser `getUserMedia` API. A two-second countdown is shown before the frame is grabbed onto an off-screen canvas at the device pixel ratio to preserve native resolution, then encoded as a PNG blob. Alternatively, an existing image file of up to 8 MB may be uploaded. In parallel, the page fetches the list of enrolled students from the campus academic information system (SIS) through an internal proxy endpoint; the operator selects the student number (NIM) from a searchable drop-down, and the student name and study program fields are filled automatically. This eliminates manual typing of identity data and the transcription errors it causes.

2) Preprocessing. The server normalizes the received image to PNG and downscales it to a width of 800 pixels while preserving the aspect ratio. This bounds the inference cost of the segmentation network and keeps the response time predictable on CPU-only servers.

3) Background removal. The preprocessed image is passed to the Rembg command-line interface, which runs U2-Net through ONNX Runtime and produces an RGBA image whose background pixels are fully transparent in the alpha channel.

4) Post-processing. The segmented image is upscaled back to the original capture width, then refined with a light Gaussian blur (sigma 0.3) followed by a 3x3 median filter. This suppresses jagged edges and halo artifacts along the matting boundary that are typical of binary-mask compositing [14], yielding smoother hair and shoulder contours.

5) Card composition. On the client, the transparent photograph is layered onto the selected card template—undergraduate (S1) or graduate (S2)—at the predefined placeholder position, together with the student name (rendered in title case), study program, and student number. The operator previews the composed card in real time and downloads the result for printing; at the same moment, the transparent master photograph is submitted to the archival endpoint.

6) Structured archival. The server stores the master photograph using the naming pattern NIM_Name.png, with whitespace converted to underscores and defensive fallbacks when a field is empty. The target directory follows the hierarchy uploads/idcards/<Year>/<Month>/<Study_Program>/ and is created automatically when it does not yet exist. Because the year, month, and study program are encoded in the path itself, the archive remains fully browsable and traceable without any database lookup.

```
// POST /api/remove-background
meta = sharp(input).metadata()
sharp(input).resize(width: 800) -> resized
exec("rembg i resized noBg") // U2-Net
sharp(noBg).resize(meta.width)
               .blur(0.3).median(3) -> final RGBA

// POST /api/save-idcard
dir = uploads/idcards/YYYY/MM/Prodi/
mkdir(dir, recursive)
file = NIM_Name.png
write(dir + file)
```

Fig. 1. Simplified core logic of the server-side processing pipeline

A deliberate design choice is that the archived artifact is the transparent master photograph rather than the flattened card image. The composed card can be re-rendered from the master at any time, so template revisions (for example, a new card design in a later academic year) do not invalidate the archive, and the same master can be reused for other academic media without repeating the segmentation step.

C. Gallery and Bulk Distribution

The gallery module scans the archive directory recursively and returns, for every stored image, its file name, study program, year, month, creation time, and size, all derived from the storage path and file metadata. The user interface presents this list with search and filter controls by period and study program. Two management operations are provided: single-file deletion, in which the proxy URL is translated back to the physical path before removal, and bulk export, in which every file matching the active filter is compressed into a single ZIP archive and downloaded for mass card printing. This closes the workflow loop from capture to distribution without leaving the application.

D. Testing Scenario

Testing was carried out on 100 sample photographs of students captured with a webcam against relatively plain backgrounds under varying lighting conditions, in order to reflect realistic field conditions. Three aspects were measured: (a) processing time per photograph compared with the manual method performed by an experienced operator using an image editor; (b) background-removal quality, assessed qualitatively by three raters into the categories Success (clean edges, subject intact), Fair (usable after minor correction), and Fail (broken edges or cropped subject); and (c) the correctness of file naming and directory placement for every archived file. In addition, black-box functional testing was performed on all main features to verify that each behaves according to its specification.

III. RESULTS AND DISCUSSION

A. System Implementation

The system was deployed as two PM2-managed services behind the campus network. The capture page presents a live camera preview beside the card preview panel; once a student is selected from the SIS-backed drop-down, capturing a photograph triggers the full pipeline and the composed card appears within seconds, ready for download. The operator can switch between the undergraduate and graduate templates with a single control, and the identity text on the card updates immediately. The gallery page lists the archive grouped by period and study program, with filtering, deletion, and one-click ZIP export of the filtered subset. In day-to-day use, the workflow for one student consists of exactly three actions: select the student number, click capture, and click download.

B. Background-Removal Quality

Table II summarizes the segmentation quality obtained on the 100 test samples. The large majority of photographs taken under adequate lighting with plain backgrounds were segmented cleanly, with intact hair and shoulder boundaries. The small number of failures was generally caused by strong shadows around the subject or clothing whose color closely resembled the

background, conditions that are known to be difficult for salient object detection in general [4], [5].

TABLE II. DISTRIBUTION OF BACKGROUND-REMOVAL QUALITY ON 100 SAMPLES

Segmentation Result	Samples	Percentage
Success (clean edges, subject intact)	86	86%
Fair (minor correction needed)	10	10%
Fail (broken edges / cropped subject)	4	4%
Total	100	100%

It should be noted that the figures in Table II were obtained from internal trial runs and should be recalibrated against local data when the system is adopted by another institution. Nevertheless, the trend confirms that the U2-Net-based approach through Rembg handles the vast majority of cases well without any manual intervention.

C. Processing-Time Efficiency

Table III compares the per-photograph processing time of the manual method and the automated system. In the manual workflow, removing the background of a single photograph in an image editor takes on average 3-5 minutes, to which naming, template composition, and archiving time must be added. The automated system completes the entire chain—from background removal to the archival of the master file—in under 6 seconds per photograph on a CPU-only server.

TABLE III. COMPARISON OF MANUAL AND AUTOMATED PROCESSING TIME

Stage	Manual (per photo)	Automated
Background removal	180-300 s	approx. 3-5 s
File naming	10-20 s	< 1 s
Card composition	60-120 s	< 1 s (live)
Storage and archiving	10-30 s	< 1 s
Total (estimate)	approx. 4-8 min	< 6 s

The efficiency gain becomes decisive at scale. Processing 1,000 new students manually would consume tens of working hours, whereas the automated system finishes the same volume within hours including the photo sessions themselves, since the computation per student is only a few seconds. Because Rembg executes fully offline through ONNX Runtime, throughput can be increased simply by scaling the local server, and no student data is exposed to third-party services—an important property for personal data such as student photographs.

D. File Naming and Storage Verification

All 100 processed files were archived with the correct name and in the correct directory. Table IV shows a concrete example of the archival result for one student. The year/month/study-program hierarchy gives administrators direct traceability: all photographs taken in a given period or belonging to a given study program can be located by simple directory navigation, with no database query required. Naming based on the student number guarantees per-

student identifiability, while the inclusion of the student name eases visual verification. The PNG format with an alpha channel was chosen for the master file because it preserves the transparent background losslessly, so the master remains reusable for future card templates and other academic media.

TABLE IV. EXAMPLE OF THE ARCHIVAL RESULT FOR ONE STUDENT

Item	Value
Name pattern	NIM_Name.png
Example file	102021001_Budi_Santoso.png
Storage path	uploads/idcards/2026/07/ Computer_Science/
Access URL	/api/gallery/image/2026/07/ Computer_Science/102021001_ Budi_Santoso.png

E. Functional Testing

Table V lists the black-box test results for the main features. All core functions behaved according to specification, including the automatic filling of student data from the academic information system, the switching of card templates, and the gallery management operations.

TABLE V. BLACK-BOX FUNCTIONAL TEST RESULTS

Test Scenario	Expected Result	Status
Fetch student list from SIS and autofill by NIM	Name and study program filled automatically	Pass
Camera capture with countdown	3:4 frame captured at native resolution	Pass
Automatic background removal	Background removed, subject intact	Pass
Template selection (S1/S2)	Card preview switches template	Pass
Card composition and download	Identity text and photo on placeholder	Pass
Automatic folder creation (year/month/program)	Directory created when absent	Pass
File naming and archival	Name follows NIM_Name pattern	Pass
Gallery list, filter, delete, and bulk ZIP export	Operations succeed on filtered files	Pass

F. Discussion

The findings are consistent with prior literature stating that image-processing technology can replace error-prone and time-consuming manual KTM workflows [1]. Compared with recognition-oriented studies such as student face recognition [10] and OCR-based KTM information extraction [1], which operate on the downstream side of existing images, the present system addresses the upstream production side—generating and archiving the visual artifact itself—so the two lines of work are complementary and can be integrated within a larger academic information system.

Three properties distinguish the proposed system from conventional practice. First, background removal is fully automatic and based on U2-Net [7], requiring no skilled operator; the operator workload per student

drops to three clicks. Second, identity data is bound directly from the academic information system rather than typed, which removes an entire class of transcription errors from the printed cards. Third, the archive is structured by construction: every file is named and placed deterministically, so traceability does not depend on operator discipline.

The system also has limitations. Segmentation quality still depends on capture conditions; strong shadows or clothing colors similar to the background can degrade edge accuracy, as is typical for salient object detection [4]. The current deployment uses the general-purpose U2-Net weights rather than the human-segmentation variant available in Rembg, which slightly sacrifices portrait-specific accuracy for generality; switching models is a one-line configuration change. Retaking a photograph within the same period overwrites the previous master file, which guarantees a single canonical photo per period but leaves cross-period duplicates to be managed manually through the gallery. Finally, archival metadata lives entirely in the directory structure; while this keeps the system dependency-free, integrating a database layer would enable richer queries such as per-cohort completeness reports.

Future work can address these limitations by adopting more recent segmentation models such as IS-Net for dichotomous image segmentation [15] or BiRefNet [16] for high-resolution mattes, or promptable segmentation models such as the Segment Anything Model [17] for difficult cases; by adding face detection to validate pose and center the portrait automatically before composition; and by recording archival events in the institutional database so that the card-production pipeline is fully connected to student records.

IV. CONCLUSION

This research has designed and built a web-based system that automates student identity card photo processing end to end, integrating AI background removal (Rembg/U2-Net through ONNX Runtime), browser-based photo capture, automatic student-data binding from the campus academic information system, dynamic card composition on undergraduate and graduate templates, structured archival in a year/month/study-program hierarchy with student-number-based naming, and a gallery module with filtering, deletion, and bulk ZIP export for mass printing.

Testing on 100 sample photographs shows that the system reduces per-student processing time from roughly 3-5 minutes with the manual method to under 6 seconds, achieves clean segmentation on 86% of samples without manual correction under adequate lighting, and archives every file with correct naming and placement. All main functions passed black-box testing. The proposed system therefore improves the efficiency, consistency, and traceability of KTM photo management while keeping all personal data on-premises.

Future research will focus on stronger segmentation models for difficult lighting conditions, automatic face-based pose validation and portrait alignment, and direct integration with the institutional student database so that card production becomes a fully connected part of the academic information system.

REFERENCES

- [1] Y. Reswan, R. Raffles, A. Wijaya, and Y. Apridiansyah, "Penerapan algoritma OCR untuk ekstraksi informasi dari citra kartu tanda mahasiswa (KTM)," *JATI (Jurnal Mahasiswa Teknik Informatika)*, vol. 8, no. 5, Oct. 2024, doi: 10.36040/jati.v8i5.11006.
- [2] J. Long, E. Shelhamer, and T. Darrell, "Fully convolutional networks for semantic segmentation," in *Proc. IEEE Conf. Computer Vision and Pattern Recognition (CVPR)*, 2015, pp. 3431-3440.
- [3] O. Ronneberger, P. Fischer, and T. Brox, "U-Net: Convolutional networks for biomedical image segmentation," in *Proc. Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, LNCS 9351, 2015, pp. 234-241, doi: 10.1007/978-3-319-24574-4_28.
- [4] A. Borji, M.-M. Cheng, Q. Hou, H. Jiang, and J. Li, "Salient object detection: A survey," *Computational Visual Media*, vol. 5, no. 2, pp. 117-150, 2019.
- [5] W. Wang, Q. Lai, H. Fu, J. Shen, H. Ling, and R. Yang, "Salient object detection in the deep learning era: An in-depth survey," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 44, no. 6, pp. 3239-3259, 2022.
- [6] S. Minaee, Y. Boykov, F. Porikli, A. Plaza, N. Kehtamavaz, and D. Terzopoulos, "Image segmentation using deep learning: A survey," *IEEE Trans. Pattern Analysis and Machine Intelligence*, vol. 44, no. 7, pp. 3523-3542, 2022.
- [7] X. Qin, Z. Zhang, C. Huang, M. Dehghan, O. R. Zaiane, and M. Jagersand, "U2-Net: Going deeper with nested U-structure for salient object detection," *Pattern Recognition*, vol. 106, p. 107404, 2020, doi: 10.1016/j.patcog.2020.107404.
- [8] D. Gatis, "Rembg: A tool to remove images background," *GitHub repository*, 2024. [Online]. Available: <https://github.com/danielgatis/rembg>
- [9] ONNX Runtime developers, "ONNX Runtime," 2024. [Online]. Available: <https://onnxruntime.ai>
- [10] I. P. Sari, F. Ramadhani, A. Satria, and D. Apdilah, "Implementasi pengolahan citra digital dalam pengenalan wajah menggunakan algoritma PCA dan Viola Jones," *Hello World: Jurnal Ilmu Komputer*, vol. 2, no. 3, pp. 146-157, 2023, doi: 10.56211/helloworld.v2i3.346.
- [11] R. T. Fielding and R. N. Taylor, "Principled design of the modern Web architecture," *ACM Trans. Internet Technology*, vol. 2, no. 2, pp. 115-150, 2002.
- [12] L. Fuller, "sharp: High performance Node.js image processing," 2025. [Online]. Available: <https://sharp.pixelplumbing.com>
- [13] Vercel Inc., "Next.js documentation," 2025. [Online]. Available: <https://nextjs.org/docs>
- [14] T. Porter and T. Duff, "Compositing digital images," *ACM SIGGRAPH Computer Graphics*, vol. 18, no. 3, pp. 253-259, 1984.
- [15] X. Qin, H. Dai, X. Hu, D.-P. Fan, L. Shao, and L. Van Gool, "Highly accurate dichotomous image segmentation," in *Proc. European Conf. Computer Vision (ECCV)*, 2022, pp. 38-56.
- [16] P. Zheng et al., "Bilateral reference for high-resolution dichotomous image segmentation," *CAAI Artificial Intelligence Research*, vol. 3, art. 9150038, 2024.
- [17] A. Kirillov et al., "Segment anything," in *Proc. IEEE/CVF Int. Conf. Computer Vision (ICCV)*, 2023, pp. 4015-4026.

